MCMP117 · Week 8 · Study Guide

Fundamentals of Operating Systems

Midterm Review & Practice: Weeks 1–7

Everything you need for the midterm in one place: summaries, formulas, worked problems and a full practice exam.

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain every key term of Weeks 1–7 in one or two sentences, with a concrete example.
2. Draw the standard diagrams from memory: the five-state process diagram, the system-call path, the paging hardware with a TLB, the page-fault steps, the directory structures.
3. Solve every numerical question type of the course: utilisation and overhead formulas, `fork()` counting, Amdahl's law, Gantt charts with waiting/turnaround/response times, placement with first-, best- and worst-fit, address translation, effective access time (TLB and page faults), page-replacement frame tables and direct-access block arithmetic.
4. State the course conventions (scheduling ties, priority order, Round Robin arrivals, the TLB formula) and use them consistently.
5. Write a complete 90-minute practice exam and mark it with the worked solutions.

Contents

1 The Midterm Exam 1

2 Week 1: Introduction and History 3

3 Week 2: Definitions, Goals, Components and Structures 4

4 Week 3: OS Services, System Calls and Processes 5

5 Week 4: Threads and CPU-Scheduling Basics 6

6 Week 5: Scheduling Algorithms and Synchronisation 7

7 Week 6: Memory Management 10

8 Week 7: Virtual Memory and File Systems 12

9 Common Mistakes and Exam Strategy 15

10 Formula Sheet 16

11 Practice Exam 17

12 Practice Exam: Worked Solutions 18

13 How the Numbers Were Checked 21

14 Looking Ahead: Week 9 22

1 The Midterm Exam

1.1 Logistics

Items in square brackets will be confirmed by the instructor on the course Telegram channel; check it before the exam day.

Item	Details
Date and time	[to be announced]
Room	[to be announced]
Duration	[90] minutes
Covers	Weeks 1–7 (lectures, slides and course notes)
Allowed materials	pens; [a basic, non-programmable calculator — to be confirmed]; [closed book, no notes, no phones — to be confirmed]; [formula sheet provided: yes/no — to be confirmed]
Bring	your student ID; arrive 10 minutes early

1.2 Format

The sample paper distributed with this week's material (`exam/MCMP117_Midterm_Sample.pdf`) and the practice exam in Section 11 both follow this structure; the real paper follows it unless announced otherwise.

Part	Question type	Points	Suggested time
A	10 multiple-choice questions, 2 points each	20	15 min
B	4 short-answer questions, 5 points each	20	20 min
C	5 numerical problems	60	45 min
	reading at the start, checking at the end		$5 + 5$ min
	Total	**100**	**[90] min**

With 90 minutes for 100 points you have about 0.9 minutes per point: a 2-point question should not take 10 minutes.

1.3 Course conventions: state them in your answers

Textbooks differ in small details, so the course fixed the following conventions (Weeks 5 and 6). Write the relevant one at the top of a numerical answer.

Conventions used in MCMP117

- **Scheduling:** context-switch time is taken as 0 (switches are counted separately); ties are broken by arrival time, then by process name; a *smaller* priority number means a *higher* priority; in Round Robin a process that arrives at the same instant as a quantum expires enters the ready queue *before* the preempted process.
- **TLB:** the lookup is counted on every access and a miss costs one extra memory access: EAT $= t_{\text{TLB}} + (2 - \alpha)\, t_m$.
- **Placement:** holes are listed in address order; a request is carved from the beginning of the chosen hole.
- **Page replacement:** all frames start empty; the new page goes into the victim's frame.

1.4 How to use this study guide

Sections 2–8 give, for each week, a short summary, the formulas, the typical exam questions and a worked example; the worked examples are the problems solved on this week's slides. Section 10 is a one-page formula sheet. Section 11 is a complete practice exam: write it in 90 minutes *without* looking at anything, then mark it with the solutions in Section 12. Every number in this guide was computed by the Python scripts in the `code/` folder (Section 13).

2 Week 1: Introduction and History

Summary

A computer follows instructions: input $\rightarrow$ process $\rightarrow$ output, with the program stored in memory (the **stored-program** idea). A first working definition: the OS is the program that starts first and manages everything else, so that other programs can run; it shares the hardware safely and hides its hard parts. History

moved through generations, each solving the problem of an expensive *idle* processor:

- **No OS** (ENIAC, 1945): one job at a time, programmed by rewiring plugboards.
- **Batch processing** (GM-NAA I/O, 1956, often called the first OS): a **resident monitor** loads and runs jobs one after another, removing the human set-up time.
- **Multiprogramming** (1960s): several jobs in memory; when one waits for I/O, the CPU switches to another.
- **Time-sharing** (CTSS at MIT, 1961): each user gets a short **time quantum** in rotation and experiences an interactive machine.
- IBM System/360 and OS/360 (1964): one OS for a family of machines. Multics (1965–69) inspired **UNIX** (Bell Labs, 1969; rewritten in C in 1973, hence portable).
- Personal computers (Apple II 1977, IBM PC and MS-DOS 1981, Macintosh 1984, Windows 95), **Linux** (1991, free UNIX-like kernel; with GNU a complete free OS), smartphones (iPhone 2007, Android 2008 on a Linux kernel), the cloud (AWS 2006, virtual machines, containers).

Week 1 formulas

$$U = \frac{t_{\text{run}}}{t_{\text{run}} + t_{\text{setup}}}, \qquad U(n) = 1 - p^n, \qquad T_{\text{wait}} \approx n\,q.$$

U: CPU utilisation; p: fraction of time a job waits for I/O; n: number of jobs in memory (or users); q: time quantum.

Typical questions

Match milestones with the idea they introduced (multiple choice); explain the difference between multiprogramming (CPU efficiency) and time-sharing (user experience); explain why rewriting UNIX in C mattered; small utilisation calculations.

Worked problem 1(a) of the slides: multiprogramming

Jobs wait for I/O a fraction $p = 0.6$ of the time. Then $U(1) = 1 - 0.6 = 40\%$, $U(3) = 1 - 0.6^3 = 1 - 0.216 = 78.4\%$ and $U(6) = 1 - 0.6^6 \approx 95.3\%$. The smallest n with $U \geq 95\%$ needs $0.6^n \leq 0.05$, i.e. $n \geq \ln 0.05 / \ln 0.6 \approx 5.86$, so $n = 6$ (check: $0.6^5 \approx 0.078 > 0.05$, $0.6^6 \approx 0.047 \leq 0.05$).
Batch: with $t_{\text{run}} = 4$ min and $t_{\text{setup}} = 6$ min, $U = 4/(4+6) = 40\%$; a monitor that cuts the set-up to 1 min gives $U = 4/5 = 80\%$. **Time-sharing:** 30 users with $q = 20$ ms wait at most about $n\,q = 30 \times 20$ ms $= 600$ ms.

3 Week 2: Definitions, Goals, Components and Structures

Summary

- **Working definition:** the OS is the software layer that manages the hardware and offers programs a simpler, safer machine; its always-resident core is the **kernel**. Kernel, system programs (shell, `ls`, compiler) and application programs are different layers.

- **Two views:** top-down, the OS is an *extended machine* that offers clean abstractions (files instead of sectors, processes instead of registers); bottom-up, it is a *resource manager* that multiplexes resources in time (CPU, printer) and in space (memory, disk).
- **Design goals:** convenience, efficiency and the ability to evolve; they conflict and are weighted differently on PCs, servers, phones, embedded and real-time systems.
- **Four components:** hardware, operating system, application programs, users.
- **Interrupts:** a device controller signals the CPU, which saves the state, runs the handler found in the **interrupt vector**, and resumes; the OS is interrupt-driven. Polling wastes CPU time.
- **Storage hierarchy:** registers (≈ 0.3 ns), cache, RAM (≈ 100 ns), SSD (≈ 50 μs), HDD (≈ 5 ms); registers, cache and RAM are volatile.
- **Dual-mode operation:** mode bit 0 = kernel mode, 1 = user mode; a privileged instruction in user mode traps to the OS; a **system call** switches to kernel mode and back. A measured system call cost ≈ 116 ns, a function call ≈ 1.5 ns.
- **Structures:** simple (MS-DOS, no protection), monolithic (UNIX, Linux: fast, fragile), layered (THE: easy to debug, slow), microkernel (MINIX 3, QNX, seL4: reliable, slower), modular (Linux loadable kernel modules), hybrid (Windows, macOS, Android).

Week 2 formulas

$$\text{polling share} = f \cdot t_{\text{check}}, \qquad T_{\text{mono}} = 2t_s + t_w, \qquad T_{\mu} = 4t_s + 2t_c + t_w.$$

t_s: one mode switch; t_c: one message copy; t_w: useful work.

Typical questions

Explain the two views with an example each; which design goal matters most for a phone, a server, a brake controller; what happens when a user program executes a privileged instruction; difference between `printf()` and `write()`; one advantage and one disadvantage of a monolithic kernel and of a microkernel.

Microkernel cost and polling

With $t_s = 0.1$ μs, $t_c = 0.15$ μs and $t_w = 2$ μs: $T_{\text{mono}} = 0.2 + 2 = 2.2$ μs, $T_{\mu} = 0.4 + 0.3 + 2 = 2.7$ μs, so the microkernel is $2.7/2.2 \approx 1.23$ times slower for this request; the longer the useful work, the closer the ratio comes to 1. Polling a device $f = 200$ times per second at $t_{\text{check}} = 5$ μs costs $200 \times 5 \times 10^{-6} = 10^{-3} = 0.1\%$ of the CPU.

4 Week 3: OS Services, System Calls and Processes

Summary

- **Services** for the user: user interface, program execution, I/O operations, file-system manipulation, communication (shared memory or message passing), error detection. For the system itself: resource allocation, accounting and logging, protection and security.

- **API, wrapper, system call:** `printf()` is an API function; the `write()` wrapper puts the system-call number in `rax` and the parameters in registers and executes `syscall`; the kernel checks the number, looks it up in the **system-call table** and calls `sys_write()`. `strlen()` never makes a system call.
- **Parameters** are passed in registers, in a memory block whose address is in a register, or on the stack. Six categories: process control, file management, device management, information maintenance, communication, protection.
- A **process** is a program in execution, with text, data, bss, heap and stack regions. Five states: new, ready, running, waiting, terminated; there is no arrow from waiting to running.
- The **PCB** holds state, PID/PPID, program counter, registers, scheduling, memory-management, accounting and I/O status information. A **context switch** saves the old PCB and loads the new one; it is pure overhead.
- `fork()` returns 0 in the child and the child's PID in the parent; `exec()` replaces the program (same PID); `wait()` collects the child. A **zombie** has terminated but has not been waited for; an **orphan**'s parent terminated first (Linux re-parents it to PID 1).

Week 3 formulas

$$\text{context-switch overhead} = \frac{s}{q+s};$$

n calls of `fork()`, each executed by every existing process, give 2^n processes ($2^n - 1$ of them created).

Typical questions

Draw the five-state diagram and name the event behind each transition; list PCB fields; classify system calls into the six categories; what `fork()` returns; zombie versus orphan; overhead and `fork()` counting.

Worked problems 1(b) and 1(c) of the slides

Overhead. With $s = 4$ μs: for $q = 2$ ms $= 2000$ μs the overhead is $4/2004 \approx 0.20\%$; for $q = 50$ μs it is $4/54 \approx 7.4\%$. Tiny quanta waste the CPU.
fork(). `fork(); fork(); printf("A\n"); fork(); printf("B\n");` Every existing process executes every later `fork()`. Before the first `printf` there are $2 \times 2 = 4$ processes, so A is printed 4 times; the third fork doubles them to 8 and B is printed 8 times; 8 processes in total, 7 created. (A real run with `os.fork()` in `review_problems.py` printed exactly 4 A and 8 B. Exam questions assume output to a terminal; with output redirected to a file, `printf`'s buffer is copied by `fork()`, as shown in Week 3.)

5 Week 4: Threads and CPU-Scheduling Basics

Summary

- A **thread** has its own thread ID, program counter, registers and stack; it shares the code, global data, heap and open files of its process. Benefits: responsiveness, resource sharing, economy (a thread was about 4 times cheaper to create than a process on our machine), scalability.

- **Concurrency**: several tasks make progress; **parallelism**: several tasks run at the same instant (needs at least two cores).
- User-level threads (fast, but a blocking call blocks the whole process) versus kernel-level threads. Models: many-to-one, one-to-one (Linux, Windows), many-to-many (Go, Java virtual threads).
- Processes alternate CPU and I/O bursts (CPU-bound versus I/O-bound). The short-term scheduler chooses from the ready queue at four decision points (running→waiting, running→ready, waiting→ready, termination); non-preemptive scheduling uses only the first and the last. The **dispatcher** switches context, switches to user mode and jumps to the saved PC; its time is the dispatch latency.

Week 4 formulas

$$\text{speedup}(N) \le \frac{1}{S + \dfrac{1-S}{N}}, \qquad \lim_{N\to\infty} \text{speedup} = \frac{1}{S};$$

$$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}, \quad T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}}, \quad T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}},$$

$$\text{CPU utilisation} = \frac{B}{T}, \qquad \text{throughput} = \frac{n}{T}.$$

Maximise utilisation and throughput; minimise turnaround, waiting and response time.

Typical questions

Which items are shared and which are private (the stack is private, the heap is shared); which model Linux uses; concurrency versus parallelism; Amdahl speedups; the four decision points; computing the criteria from a Gantt chart.

Worked problem 1(d) of the slides: Amdahl's law

With $S = 0.2$: $N = 2$: $1/(0.2 + 0.4) \approx 1.67$; $N = 4$: $1/(0.2 + 0.2) = 2.5$; $N = 8$: $1/(0.2 + 0.1) \approx 3.33$; the limit is $1/0.2 = 5$. Eight cores give only 3.33. Conversely, a speedup of at least 3 on 4 cores requires $S + (1 - S)/4 \le 1/3$, i.e. $S \le 1/9 \approx 0.11$.

6 Week 5: Scheduling Algorithms and Synchronisation

Summary: scheduling

Algorithm	Rule	Preemptive	Remember
FCFS	FIFO ready queue; run to completion	no	convoy effect
SJF	shortest next CPU burst first	no	optimal average wait when all are ready; burst must be predicted
SRTF	a newcomer shorter than the *remaining* time preempts	yes	minimum average wait; long jobs may starve
Priority	highest priority (smallest number) first	either	starvation; cure: aging
Round Robin	quantum q; preempted process to the tail	yes	q too large $\rightarrow$ FCFS; too small $\rightarrow$ overhead

Linux schedules normal tasks for fairness (CFS, EEVDF since kernel 6.6), weighted by the nice value; multi-level feedback queues approximate SJF by watching behaviour.

Summary: synchronisation

- A **race condition** occurs when several threads access the same data concurrently, at least one writes, and the outcome depends on the order; `counter++` is load–add–store.
- The **critical-section problem**: entry, critical, exit and remainder sections; a solution needs *mutual exclusion*, *progress* and *bounded waiting*.
- Disabling interrupts fails on multiprocessors and is privileged; Peterson's solution (two threads, needs memory barriers on modern CPUs); atomic `test_and_set` and compare-and-swap; **mutex locks** (`acquire/release`; spinlock versus sleeping).
- A **semaphore** S is accessed only by `wait(S)` (P: `while (S <= 0); S--;`) and `signal(S)` (V: S++); binary (initial 1, like a mutex) or counting (initial N). Bounded buffer: `empty` $= n$, `full` $= 0$, `mutex` $= 1$, with `wait(empty)` *before* `wait(mutex)`.
- **Deadlock**: every thread of a group waits for an event only another member can cause; prevent it by acquiring locks in one global order. Starvation is different: others do make progress.

Week 5 formulas

$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\, \tau_n,$ RR: wait for the next turn $\leq (n-1)\, q,$ switch overhead $= \dfrac{s}{q+s}.$

Typical questions

Gantt chart plus a table of finish, turnaround, waiting and response times and their averages for two or three algorithms; exponential averaging; a losing interleaving of `counter++`; the three requirements; semaphore traces; why the order of the two `wait` calls in the producer matters.

6.1 Worked problem 2 of the slides: one job set, five schedulers

Jobs (times in ms): P1 (arrival 0, burst 7, priority 2), P2 (2, 4, 1), P3 (3, 2, 4), P4 (5, 5, 3). The total work is 18 ms and the CPU is never idle. Generated tables:

FCFS



Process	Arrival	Burst	Priority	Finish	Turnaround	Waiting	Response
P1	0	7	2	7	$7-0=7$	$7-7=0$	0
P2	2	4	1	11	$11-2=9$	$9-4=5$	5
P3	3	2	4	13	$13-3=10$	$10-2=8$	8
P4	5	5	3	18	$18-5=13$	$13-5=8$	8
Average					$39/4=9.75$	$21/4=5.25$	$21/4=5.25$

SJF (non-preemptive)



At $t = 7$ the ready queue holds P2 (4), P3 (2) and P4 (5): P3 is shortest.

Process	Arrival	Burst	Priority	Finish	Turnaround	Waiting	Response
P1	0	7	2	7	$7-0=7$	$7-7=0$	0
P2	2	4	1	13	$13-2=11$	$11-4=7$	7
P3	3	2	4	9	$9-3=6$	$6-2=4$	4
P4	5	5	3	18	$18-5=13$	$13-5=8$	8
Average					$37/4=9.25$	$19/4=4.75$	$19/4=4.75$

SRTF



$t = 2$: P2 needs $4 <$ P1's remaining 5: preempt. $t = 3$: P3 needs $2 <$ P2's remaining 3: preempt. $t = 5$: remaining P1 5, P2 3, P4 5: P2 runs. $t = 8$: P1 and P4 both need 5: the tie goes to the earlier arrival, P1.

Process	Arrival	Burst	Priority	Finish	Turnaround	Waiting	Response
P1	0	7	2	13	$13 - 0 = 13$	$13 - 7 = 6$	0
P2	2	4	1	8	$8 - 2 = 6$	$6 - 4 = 2$	0
P3	3	2	4	5	$5 - 3 = 2$	$2 - 2 = 0$	0
P4	5	5	3	18	$18 - 5 = 13$	$13 - 5 = 8$	8
Average					$34/4 = 8.5$	$16/4 = 4$	$8/4 = 2$

Priority (preemptive)



P2 (priority 1) takes the CPU from P1 (priority 2) at $t = 2$; P3 has the lowest priority and runs last.

Process	Arrival	Burst	Priority	Finish	Turnaround	Waiting	Response
P1	0	7	2	11	$11 - 0 = 11$	$11 - 7 = 4$	0
P2	2	4	1	6	$6 - 2 = 4$	$4 - 4 = 0$	0
P3	3	2	4	18	$18 - 3 = 15$	$15 - 2 = 13$	13
P4	5	5	3	16	$16 - 5 = 11$	$11 - 5 = 6$	6
Average					$41/4 = 10.25$	$23/4 = 5.75$	$19/4 = 4.75$

Round Robin, $q = 3$



$t = 3$: P3 arrives exactly when P1's quantum expires, so the queue is P2, P3, P1. $t = 6$: P4 (arrived at 5) is behind P1, and P2 (1 ms left) joins the tail: P3, P1, P4, P2. $t = 8$: P3 done; P1 runs 8–11 and has 1 ms left; then P4 11–14, P2 14–15 (done), P1 15–16 (done), P4 16–18.

Process	Arrival	Burst	Priority	Finish	Turnaround	Waiting	Response
P1	0	7	2	16	$16 - 0 = 16$	$16 - 7 = 9$	0
P2	2	4	1	15	$15 - 2 = 13$	$13 - 4 = 9$	1
P3	3	2	4	8	$8 - 3 = 5$	$5 - 2 = 3$	3
P4	5	5	3	18	$18 - 5 = 13$	$13 - 5 = 8$	6
Average					$47/4 = 11.75$	$29/4 = 7.25$	$10/4 = 2.5$

Algorithm	Preemptive	Avg waiting	Avg turnaround	Avg response	Switches
FCFS	no	5.25	9.75	5.25	3
SJF	no	4.75	9.25	4.75	3
SRTF	yes	4	8.5	2	5
Priority	yes	5.75	10.25	4.75	4
RR, $q = 3$	yes	7.25	11.75	2.5	7

SRTF gives the smallest average waiting time, as the theory promises; Priority serves importance (P3 waits

13 ms); RR has the largest average wait but a good average response time.

Exponential averaging

$\alpha = 0.5$, $\tau_0 = 8$ ms, measured bursts $t_0, t_1, t_2 = 6, 10, 4$ ms: $\tau_1 = 0.5 \cdot 6 + 0.5 \cdot 8 = 7$, $\tau_2 = 0.5 \cdot 10 + 0.5 \cdot 7 = 8.5$, $\tau_3 = 0.5 \cdot 4 + 0.5 \cdot 8.5 = 6.25$ ms.

6.2 Worked problem 3 of the slides: a race and a semaphore

`counter` $= 10$; thread A executes `counter++`, thread B `counter--`, each as load–add–store into its own register. One unlucky order:

Step	Thread A	Thread B	counter
1	load (reg $= 10$)		10
2		load (reg $= 10$)	10
3	add 1 (reg $= 11$)		10
4		add -1 (reg $= 9$)	10
5	store		11
6		store	9 (should be 10)

There are $\binom{6}{3} = 20$ interleavings that keep each thread's own order. Enumerating them all (`review_problems.py`) gives the final value 9 in 9 orders, 10 in 2 orders and 11 in 9 orders: only 2 of 20 orders are correct. The fix is a critical section protected by a mutex.

Counting semaphore $S = 2$ (two printers), threads T1, T2, T3: T1 `wait` $\rightarrow$ enters, $S = 1$; T2 `wait` $\rightarrow$ enters, $S = 0$; T3 `wait` $\rightarrow$ blocks; T1 `signal` $\rightarrow S = 1$, and T3 immediately passes its `wait`, $S = 0$; T2 `signal` $\rightarrow S = 1$; T3 `signal` $\rightarrow S = 2$. At most two threads use a printer at any time.

7 Week 6: Memory Management

Summary

- **Address binding** at compile time (absolute code), load time (relocatable code) or execution time (needs the **MMU**; all modern systems). Logical (virtual) addresses are generated by the CPU; physical addresses are seen by memory.
- **Base and limit:** the MMU checks $0 \leq a <$ limit and adds the base; otherwise it traps. Loading these registers is privileged.
- **Swapping** whole processes takes seconds; modern systems swap pages.
- **Contiguous allocation** with variable partitions: first-fit, best-fit, worst-fit. **External fragmentation**: enough free memory, but not contiguous (50-percent rule: up to one third lost); **internal fragmentation**: unused memory inside an allocated block. Compaction needs execution-time binding.
- **Paging:** frames (physical) and pages (logical) of equal size 2^n; the page table maps $p \rightarrow f$; no external fragmentation, internal fragmentation of half a page per process on average; valid–invalid and protection bits; shared (reentrant) code pages; multi-level page tables.

- **TLB:** a small associative cache of (p, f) pairs; high hit ratios thanks to locality.
- **Segmentation:** address $\langle s, d \rangle$, segment table of (base, limit) pairs; variable sizes bring back external fragmentation. x86-64 Linux uses flat segments and does all translation with paging.

Week 6 formulas

$$0 \le a_{\text{log}} < \text{limit} \;\Rightarrow\; a_{\text{phys}} = \text{base} + a_{\text{log}}, \text{ otherwise trap}; \qquad t_{\text{swap}} = \frac{\text{size}}{\text{transfer rate}};$$

$$p = \left\lfloor \frac{a}{2^n} \right\rfloor, \quad d = a \bmod 2^n, \quad a_{\text{phys}} = f \cdot 2^n + d; \qquad \text{entries} = \frac{2^m}{2^n};$$

$$\text{EAT} = \alpha\,(t_{\text{TLB}} + t_m) + (1-\alpha)\,(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m;$$

$$\text{simplified } (t_{\text{TLB}} \approx 0)\text{: EAT} = \alpha\, t_m + (1-\alpha)\, 2t_m;$$

$$0 \le d < \text{limit}[s] \;\Rightarrow\; a_{\text{phys}} = \text{base}[s] + d, \text{ otherwise trap.}$$

Typical questions

The three binding times; who sees logical and physical addresses; base/limit translation in decimal and hexadecimal; swap times; placement with the three fits; internal versus external fragmentation; address translation through a page table; page-table size; EAT with a TLB and the hit ratio needed for a target EAT; segment-table translation; paging versus segmentation.

Base and limit, swapping

Base 12000, limit 2500: $a = 100 \to 12100$; $a = 2499 \to 14499$ (the last legal byte); $a = 2500$: **trap** (the check is strict). Swapping a 200 MB process at 80 MB/s takes $200/80 = 2.5$ s each way, 5 s for out plus in.

7.1 Worked problem 4 of the slides: first-, best- and worst-fit

Holes (size@start, KB, address order): $300@100$, $600@500$, $200@1300$, $450@1700$, $350@2400$ (total 1900 KB). Requests in order: A $= 340$, B $= 180$, C $= 420$, D $= 500$ KB. Generated table:

Request	First-fit	Best-fit	Worst-fit
A $= 340$	$600@500 \to$ leaves 260	$350@2400 \to$ leaves 10	$600@500 \to$ leaves 260
B $= 180$	$300@100 \to$ leaves 120	$200@1300 \to$ leaves 20	$450@1700 \to$ leaves 270
C $= 420$	$450@1700 \to$ leaves 30	$450@1700 \to$ leaves 30	must wait
D $= 500$	must wait	$600@500 \to$ leaves 100	must wait

First-fit: D must wait although 960 KB are free, because the largest hole is only 350 KB (external fragmentation). Best-fit places all four. Worst-fit fails twice (1380 KB free, largest hole 350 KB). One example proves nothing in general: in the sample paper first-fit wins and best-fit fails.

7.2 Worked problem 5 of the slides: paging and the TLB

16-bit addresses, 4 KB pages ($n = 12$): p is the first hex digit and d the last three. Page table: $0 \to 3$, $1 \to 8$, $2 \to 6$, $4 \to 10$, $5 \to 2$; page 3 and pages 6–15 invalid.

- `0x1C3B` $= 7227$: $p = 1$, $d =$ `0xC3B` $= 3131$; frame 8: $8 \times 4096 + 3131 = 35899 =$ `0x8C3B`.
- `0x4F0E` $= 20238$: $p = 4$, $d =$ `0xF0E` $= 3854$; frame $10 =$ `0xA`: `0xAF0E` $= 44814$.
- `0x2010`: $p = 2$, frame 6: `0x6010` $= 24592$.
- `0x3004`: $p = 3$ is invalid: **trap**, no physical address.

A process of 9000 B needs $\lceil 9000/4096 \rceil = 3$ frames ($9000 = 2 \times 4096 + 808$), so $4096 - 808 = 3288$ B are lost to internal fragmentation. With 32-bit addresses, 8 KB pages and 4-byte entries a flat page table has $2^{32}/2^{13} = 2^{19} = 524\,288$ entries, i.e. 2^{21} B $= 2$ MB.

TLB: $t_{\text{TLB}} = 10$ ns, $t_m = 80$ ns, $\alpha = 0.95$:

$$\text{EAT} = 0.95 \times 90 + 0.05 \times 170 = 85.5 + 8.5 = 94 \text{ ns} = 10 + (2 - 0.95) \times 80.$$

Without a TLB every access costs $2t_m = 160$ ns; the simplified formula gives $0.95 \times 80 + 0.05 \times 160 = 84$ ns. For EAT $= 100$ ns: $10 + (2 - \alpha)\,80 = 100 \Rightarrow 2 - \alpha = 1.125 \Rightarrow \alpha = 0.875$.

Segment table

Segment 0: base 1000, limit 800; segment 1: base 5200, limit 300; segment 2: base 2400, limit 1500. $\langle 2, 1499 \rangle$: $1499 < 1500 \Rightarrow 2400 + 1499 = 3899$. $\langle 1, 300 \rangle$: $300 \geq 300 \Rightarrow$ **trap**. $\langle 0, 799 \rangle$: $799 < 800 \Rightarrow 1000 + 799 = 1799$.

8 Week 7: Virtual Memory and File Systems

Summary: virtual memory

- **Virtual memory** keeps only the pages in use in RAM; the rest stay in the backing store. It works because of **locality of reference**.
- **Demand paging** loads a page on its first use; the valid–invalid bit says whether a page is in memory. A **page fault** is handled in six steps: reference, trap, locate (page on disk and a free frame), read, update the page table, restart the instruction. Minor faults need no disk I/O, major faults do.
- **Copy-on-write**: after `fork()` pages are shared read-only until one process writes; only the written page is copied.
- **Page replacement**: FIFO (loaded longest ago; Belady's anomaly possible), OPT (used farthest in the future; minimum faults; needs the future), LRU (used longest ago; approximated by the clock / second-chance algorithm with the reference bit). The dirty bit avoids writing back clean victims.
- **Thrashing**: processes spend more time paging than executing; the cure is to reduce the degree of multiprogramming. The **working set** WS(t, Δ) is the set of distinct pages in the last Δ references.

Summary: file systems

- A **file** has attributes (name, identifier, type, location, size, protection, timestamps) kept in the file control block (the **inode** in Linux; the name is in the directory). Operations: create, open, read, write, seek, close, delete (`unlink`), truncate. `open()` returns a file descriptor; per-process tables hold the file pointer, the system-wide table the open count.

- **Access methods**: sequential (tape model), direct (any block n; disk model), indexed (search a small index, then read one block).
- **Directories**: single-level, two-level (UFD/MFD), tree (absolute and relative paths, `.` and `..`), acyclic graph (links), general graph (cycles; avoided in practice).
- A **hard link** is another directory entry for the same inode (link count); a **symbolic link** is a small file containing a path and dangles if the target is removed.

Week 7 formulas

$$\text{EAT} = (1-p)\,t_m + p\,t_{\text{fault}};$$

$$\text{byte offset} = k\,R, \quad \text{block} = \left\lfloor \frac{kR}{B} \right\rfloor, \quad \text{offset in block} = kR - \text{block} \cdot B;$$

$$D = \sum_i \text{WSS}_i > m \;\Rightarrow\; \text{thrashing.}$$

p: page-fault rate; k: record number (from 0); R: record size; B: block size; m: available frames. Always check the record's *last* byte $kR + R - 1$ too.

Typical questions

The six page-fault steps; why the instruction is restarted; copy-on-write and `fork()`; frame tables for FIFO, LRU and OPT; Belady's anomaly; thrashing and the working set; EAT with page faults and the largest fault rate for a given slow-down; block arithmetic; compare access methods, directory structures, hard and symbolic links.

8.1 Worked problem 6 of the slides: page replacement

Reference string $1, 2, 0, 4, 2, 3, 2, 1, 2, 4, 0, 2, 0, 1$ with 3 frames, initially empty. Mint: page just loaded; pale: fault column; grey: hit. Generated tables:

FIFO: 11 faults

reference	1	2	0	4	2	3	2	1	2	4	0	2	0	1
frame 0	1	1	1	4	4	4	4	1	1	1	1	2	2	2
frame 1		2	2	2	2	3	3	3	3	4	4	4	4	1
frame 2			0	0	0	0	2	2	2	2	0	0	0	0
fault?	F	F	F	F		F	F	F		F	F	F		F

LRU: 9 faults

reference	1	2	0	4	2	3	2	1	2	4	0	2	0	1
frame 0	1	1	1	4	4	4	4	1	1	1	0	0	0	0
frame 1		2	2	2	2	2	2	2	2	2	2	2	2	2
frame 2			0	0	0	3	3	3	3	4	4	4	4	1
fault?	F	F	F	F		F		F		F	F			F

OPT: 7 faults

reference	1	2	0	4	2	3	2	1	2	4	0	2	0	1
frame 0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
frame 1		2	2	2	2	2	2	2	2	2	2	2	2	2
frame 2			0	4	4	3	3	3	3	4	0	0	0	0
fault?	F	F	F	F		F				F	F			

FIFO: a hit (the reference to 2 at position 5) does not change the order of arrival. LRU at the reference to 3: the frames hold $4, 2, 0$, and 0 was used longest ago, so the hot page 2 stays. OPT at the reference to 4: the next uses are 2 (position 5), 1 (position 8), 0 (position 11), so 0 leaves. The string has 5 distinct pages, so every algorithm has at least 5 (compulsory) faults. With 4 frames: FIFO 8, LRU 7, OPT 6 faults — no anomaly on this string.

8.2 Worked problem 7 of the slides: page-fault EAT and block arithmetic

(a) $t_m = 100$ ns, $t_{\text{fault}} = 6$ ms $= 6\,000\,000$ ns, $p = 0.0002$:

$$\text{EAT} = 0.9998 \times 100 + 0.0002 \times 6\,000\,000 = 99.98 + 1200 = 1299.98 \text{ ns},$$

about 13 times slower than RAM, with only one fault per 5000 accesses. For a slow-down below 5%: $100 + p\,(6\,000\,000 - 100) < 105 \Rightarrow p < 5/5\,999\,900 \approx 8.3 \times 10^{-7}$, at most one fault per about 1.2 million accesses.

(b) Records of $R = 120$ B, blocks of $B = 4096$ B, record $k = 750$: byte offset $750 \times 120 = 90\,000$; block $\lfloor 90\,000/4096 \rfloor = 21$; offset in block $90\,000 - 21 \times 4096 = 3984$. The last byte is $90\,119$ and $\lfloor 90\,119/4096 \rfloor = 22$: the record *crosses* a block boundary (bytes 3984–4095 of block 21 and 0–7 of block 22), so two block reads are needed. Record 40 starts at byte 4800, in block 1 at offset 704, and fits completely.

Working set

Four processes with working-set sizes $4, 6, 3, 5$ need $D = 18$ frames. With only $m = 16$ frames available, $D > m$: thrashing will occur, and the OS should suspend one process.

9 Common Mistakes and Exam Strategy

Topic	Where points are usually lost
Scheduling	Waiting time is turnaround minus burst, not the start time; SRTF compares with the *remaining* time; RR newcomers go before the preempted process; forgetting the averages.
Units	8 ms $= 8\,000\,000$ ns; 2 ms $= 2000$ μs; KB $= 1024$ B in paging questions.
Paging	The offset is never translated; an invalid page traps and has no physical address; $a =$ limit already traps (strict $<$).
Frame tables	FIFO: a hit does not change the order; LRU: a hit makes the page most recent; OPT: look forward. Count the faults twice.
Symbols	p is the I/O-wait fraction in Week 1 but the page-fault rate in Week 7; α is the averaging weight in Week 5 but the TLB hit ratio in Week 6.
`fork()`	Every existing process executes every later `fork()`.
Definitions	Concurrency $\neq$ parallelism; mutual exclusion $\neq$ progress $\neq$ bounded waiting; starvation $\neq$ deadlock; hard link $\neq$ symbolic link.

Strategy. Read the whole paper first (5 min). Start with what you know best. For every numerical answer write the formula, substitute the numbers with units, compute, and box the result: method earns partial credit even when the arithmetic slips. Draw Gantt charts and frame tables neatly, with times. If you are stuck, write the relevant formula and move on. Keep 5 minutes at the end to check units and averages.

10 Formula Sheet

Wk	Quantity	Formula
1	Batch utilisation	$U = \dfrac{t_{\text{run}}}{t_{\text{run}} + t_{\text{setup}}}$
1	Multiprogramming utilisation	$U(n) = 1 - p^n$ (p: I/O-wait fraction)
1	Time-sharing worst-case wait	$T_{\text{wait}} \approx n\,q$
2	Polling cost	$f \cdot t_{\text{check}}$
2	Monolithic vs microkernel	$T_{\text{mono}} = 2t_s + t_w$; $T_\mu = 4t_s + 2t_c + t_w$
3, 5	Context-switch overhead	$\dfrac{s}{q+s}$
3	n forks in a row	2^n processes, $2^n - 1$ created
4	Amdahl's law	speedup$(N) \le \dfrac{1}{S + (1-S)/N}$; limit $\dfrac{1}{S}$
4	Turnaround, waiting	$T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$; $T_{\text{waiting}} = T_{\text{turnaround}} - T_{\text{burst}}$
4	Response	$T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}$
4	Utilisation, throughput	$\dfrac{B}{T}$ (busy / total); $\dfrac{n}{T}$
5	Exponential average	$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n$
5	Round Robin bound	wait for the next turn $\le (n-1)\,q$
6	Base and limit	$0 \le a <$ limit $\Rightarrow a_{\text{phys}} =$ base $+ a$, else trap
6	Swap time	size / transfer rate
6	50-percent rule	$\dfrac{0.5N}{N + 0.5N} = \dfrac{1}{3}$ of memory may be lost
6	Paging	$p = \lfloor a/2^n \rfloor$, $d = a \bmod 2^n$, $a_{\text{phys}} = f \cdot 2^n + d$
6	Page-table size	$2^m/2^n$ entries $\times$ entry size
6	TLB (course convention)	EAT $= \alpha(t_{\text{TLB}} + t_m) + (1-\alpha)(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m$
6	TLB (simplified)	EAT $= \alpha\, t_m + (1-\alpha)\,2t_m$
6	Segmentation	$0 \le d <$ limit$[s] \Rightarrow$ base$[s] + d$, else trap
7	Page-fault EAT	EAT $= (1-p)\,t_m + p\,t_{\text{fault}}$ (p: fault rate)
7	Direct access, record k	offset $= kR$; block $= \lfloor kR/B \rfloor$; in block $= kR -$ block $\cdot\, B$; check byte $kR + R - 1$
7	Working set	$D = \sum_i \text{WSS}_i > m \Rightarrow$ thrashing

Powers of two: $2^{10} = 1024$, $2^{12} = 4096$, $2^{16} = 65\,536$, $2^{20} \approx 10^6$. **Hex:** one hex digit $= 4$ bits; with 4 KB pages the offset is the last three hex digits. **Time:** 1 ms $= 10^3$ μs $= 10^6$ ns.

11 Practice Exam

Instructions. 90 minutes, 100 points. Use the course conventions of Section 1. Show every step of a calculation. Solutions are in Section 12; do not look before you have finished.

Part A --- Multiple choice (10 × 2 = 20 points)

A1. GM-NAA I/O (1956) is often called the first operating system because it (a) introduced time-sharing (b) kept a resident monitor that ran jobs one after another (c) was written in C (d) had a graphical interface

A2. In dual-mode operation, kernel mode is indicated by the mode bit value (a) 0 (b) 1 (c) 2 (d) 3

A3. Which of the following is *private* to each thread of a process? (a) global variables (b) heap (c) stack (d) open files

A4. When the disk read requested by a waiting process completes, the process moves to the state (a) running (b) ready (c) new (d) terminated

A5. The convoy effect is a weakness of (a) FCFS (b) SRTF (c) Round Robin (d) priority scheduling with aging

A6. A counting semaphore initialised to 3 protects three identical printers. How many threads can be using a printer at the same time? (a) 1 (b) 2 (c) 3 (d) unlimited

A7. Paging eliminates (a) internal fragmentation (b) external fragmentation (c) both (d) neither

A8. Which page-replacement algorithm can show Belady's anomaly? (a) FIFO (b) LRU (c) OPT (d) none

A9. A hard link is (a) a file containing a path (b) another directory entry for the same inode (c) a copy of the file (d) a link that dangles when the original name is deleted

A10. `getpid()` belongs to the system-call category (a) process control (b) file management (c) information maintenance (d) protection

Part B --- Short answer (4 × 5 = 20 points)

B1. Explain the difference between a program and a process. List four kinds of information stored in a process control block.

B2. What is saved and restored during a context switch, and why is it called overhead? Compute the overhead for a switch time $s = 5\ \mu$s and a time slice $q = 10$ ms.

B3. State the three requirements of a solution to the critical-section problem. Explain briefly why `counter++` executed by two threads can lose an update.

B4. List the six steps the OS performs to handle a page fault. Why must the faulting instruction be restarted?

Part C --- Problems (60 points)

C1. **[16 pt]** Four processes arrive at a single CPU (times in ms): P1 (arrival 0, burst 8), P2 (1, 4), P3 (2, 2), P4 (4, 5). For (a) FCFS, (b) SRTF and (c) Round Robin with $q = 4$, draw the Gantt chart and compute each process's turnaround and waiting time and the average waiting time. (d) Which algorithm gives the smallest average waiting time, and why is this expected?

C2. **[12 pt]** A system uses 16-bit logical addresses and 1 KB pages. The page table of a process is: page $0 \to$ frame 7, $1 \to 2$, $2 \to 9$, $4 \to 5$; all other pages are invalid. (a) How many bits are used for the offset and for the page number? How many pages can the address space have? (b) Translate the logical addresses 2100, 4500, 3100 and 1023 (decimal). (c) With $t_{\text{TLB}} = 5$ ns, $t_m = 100$ ns and a hit ratio of 0.9, compute the effective access time. Which hit ratio gives EAT $= 110$ ns?

C3. **[12 pt]** For the reference string $4, 1, 2, 4, 3, 5, 1, 3, 2, 4, 3, 5$ and 3 initially empty frames, draw the frame tables and count the page faults for FIFO, LRU and OPT.

C4. **[10 pt]** Memory has holes (size@start, in KB, address order) $250@200$, $500@600$, $150@1300$, $400@1600$, $300@2200$. Processes A $= 210$, B $= 380$, C $= 290$, D $= 450$ KB arrive in this order. Show where each is placed with first-fit, best-fit and worst-fit. Which requests cannot be satisfied, and why?

C5. **[10 pt]** (a) $t_m = 150$ ns, $t_{\text{fault}} = 5$ ms and the page-fault rate is $p = 0.0004$. Compute the effective access time. What is the largest p that keeps the slow-down below 20%? (b) A program has a serial fraction $S = 0.3$. Compute its maximum speedup on 4 and on 10 cores, and the limit for infinitely many cores.

12 Practice Exam: Worked Solutions

Part A

A1 (b) A2 (a) A3 (c) A4 (b) A5 (a) A6 (c) A7 (b) A8 (a) A9 (b) A10 (c).

Short reasons: A3 every thread has its own chain of function calls; A4 there is no arrow from waiting to running; A7 any free frame can hold any page, but the last page is usually not full; A8 LRU and OPT are stack algorithms.

Part B

B1. A program is a passive file of instructions on disk; a process is a program in execution, an active entity with a program counter, registers, its own memory (text, data, heap, stack) and resources. One program can be the basis of many processes. PCB contents (any four): process state; PID and parent PID; program counter; CPU registers; scheduling information (priority, queue pointers); memory-management information (base/limit or page table); accounting information; I/O status (open files).

B2. The context (program counter, registers, stack pointer, state, memory-management information) of the running process is saved into its PCB and the context of the next process is loaded from its PCB. During the switch no process does useful work, so it is pure overhead. With $q = 10$ ms $= 10\,000\ \mu$s the overhead is $s/(q+s) = 5/10\,005 \approx 0.05\%$.

B3. Mutual exclusion (at most one thread in its critical section), progress (if nobody is inside, only the

threads that want to enter decide, and not forever), bounded waiting (a limit on how often others may enter before a waiting thread: no starvation). `counter++` is three instructions: load into a private register, add, store. If both threads load the same old value before either stores, the second store overwrites the first and one update is lost.

B4. (1) the process references a page whose entry is invalid; (2) the MMU traps to the OS, which checks that the address is legal; (3) the OS locates the page on the backing store and finds a free frame (or replaces a page); (4) it reads the page from disk into the frame (meanwhile other processes run); (5) it updates the page table (frame number, bit v); (6) it restarts the instruction. The instruction did not complete when the fault occurred, so it must be executed again from the beginning, this time with the page in memory.

Part C

C1. *(Generated tables; conventions: switch time 0, ties by arrival.)*

(a) FCFS



Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	8	8	$8-0=8$	$8-8=0$	0
P2	1	4	12	$12-1=11$	$11-4=7$	7
P3	2	2	14	$14-2=12$	$12-2=10$	10
P4	4	5	19	$19-4=15$	$15-5=10$	10
Average				$46/4=11.5$	$27/4=6.75$	$27/4=6.75$

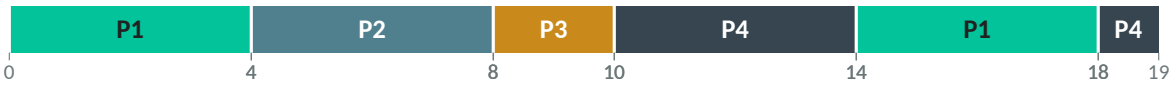
(b) SRTF



$t=1$: P2 needs $4<$ P1's remaining 7: preempt. $t=2$: P3 needs $2<$ P2's remaining 3: preempt. $t=4$: P3 done; P4 arrives needing 5; remaining P1 7, P2 3: P2 runs to 7, then P4 ($5<7$) to 12, then P1 to 19.

Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	8	19	$19-0=19$	$19-8=11$	0
P2	1	4	7	$7-1=6$	$6-4=2$	0
P3	2	2	4	$4-2=2$	$2-2=0$	0
P4	4	5	12	$12-4=8$	$8-5=3$	3
Average				$35/4=8.75$	$16/4=4$	$3/4=0.75$

(c) Round Robin, $q = 4$



$t = 4$: queue P2, P3, then P4 (arrives exactly as P1's quantum expires), then P1. P2 runs 4–8 (done), P3 8–10 (done), P4 10–14 (1 left), P1 14–18 (done), P4 18–19.

Process	Arrival	Burst	Finish	Turnaround	Waiting	Response
P1	0	8	18	$18 - 0 = 18$	$18 - 8 = 10$	0
P2	1	4	8	$8 - 1 = 7$	$7 - 4 = 3$	3
P3	2	2	10	$10 - 2 = 8$	$8 - 2 = 6$	6
P4	4	5	19	$19 - 4 = 15$	$15 - 5 = 10$	6
Average				$48/4 = 12$	$29/4 = 7.25$	$15/4 = 3.75$

(d) SRTF: average waiting time 4 ms, against 6.75 (FCFS) and 7.25 ms (RR). SRTF always gives the minimum average waiting time when context switches are free, because running a shorter job first never increases the total waiting time (exchange argument). Its price: it needs the burst lengths in advance, and the long job P1 waits 11 ms.

Rubric: each algorithm 5 points (Gantt chart 2, per-process times 2, average 1); (d) 1 point.

C2. (a) 1 KB $= 2^{10}$ B, so the offset has $n = 10$ bits and the page number $16 - 10 = 6$ bits: $2^6 = 64$ pages. (b) $p = \lfloor a/1024 \rfloor$, $d = a - 1024\,p$:

- 2100: $p = 2$, $d = 2100 - 2048 = 52$; frame 9: $9 \times 1024 + 52 = 9268$.
- 4500: $p = 4$, $d = 4500 - 4096 = 404$; frame 5: $5 \times 1024 + 404 = 5524$.
- 3100: $p = 3$ ($d = 28$): page 3 is invalid $\Rightarrow$ **trap**.
- 1023: $p = 0$, $d = 1023$; frame 7: $7 \times 1024 + 1023 = 8191$ (the last byte of frame 7).

(c) EAT $= 0.9 \times 105 + 0.1 \times 205 = 94.5 + 20.5 = 115$ ns $= 5 + (2 - 0.9) \times 100$. For 110 ns: $5 + (2 - \alpha)\,100 = 110 \Rightarrow 2 - \alpha = 1.05 \Rightarrow \alpha = 0.95$. *Rubric:* (a) 3, (b) 1.5 per address, (c) 3.

C3. *(Generated tables.)*

FIFO: 10 faults

reference	4	1	2	4	3	5	1	3	2	4	3	5
frame 0	**4**	4	4	4	**3**	3	3	3	**2**	2	2	**5**
frame 1		**1**	1	1	1	**5**	5	5	5	**4**	4	4
frame 2			**2**	2	2	2	**1**	1	1	1	**3**	3
fault?	F	F	F		F	F	F		F	F	F	F

LRU: 9 faults

reference	4	1	2	4	3	5	1	3	2	4	3	5
frame 0	**4**	4	4	4	4	4	**1**	1	1	**4**	4	4
frame 1		**1**	1	1	**3**	3	3	3	3	3	3	3
frame 2			**2**	2	2	**5**	5	5	**2**	2	2	**5**
fault?	F	F	F		F	F	F		F	F		F

OPT: 7 faults

reference	4	1	2	4	3	5	1	3	2	4	3	5
frame 0	**4**	4	4	4	**3**	3	3	3	3	3	3	3
frame 1		**1**	1	1	1	1	1	1	**2**	**4**	4	4
frame 2			**2**	2	2	**5**	5	5	5	5	5	5
fault?	F	F	F		F	F			F	F		

Check: the string has 5 distinct pages, so at least 5 faults; OPT $\leq$ LRU $\leq$ FIFO here. *Rubric:* 4 points per algorithm (3 for the table, 1 for the count).

C4. *(Generated table.)*

Request	First-fit	Best-fit	Worst-fit
A $= 210$	$250@200 \to$ leaves 40	$250@200 \to$ leaves 40	$500@600 \to$ leaves 290
B $= 380$	$500@600 \to$ leaves 120	$400@1600 \to$ leaves 20	$400@1600 \to$ leaves 20
C $= 290$	$400@1600 \to$ leaves 110	$300@2200 \to$ leaves 10	$300@2200 \to$ leaves 10
D $= 450$	must wait	$500@600 \to$ leaves 50	must wait

First-fit: D (450) must wait although 720 KB are free — the largest hole is 300 KB. Worst-fit: D must wait as well (720 KB free, largest hole 290 KB). Best-fit places all four, leaving 270 KB in small holes. This is external fragmentation: enough free memory in total, but not contiguous. *Rubric:* 3 points per strategy, 1 for the explanation.

C5. (a) $t_{\text{fault}} = 5$ ms $= 5\,000\,000$ ns:

$$\text{EAT} = 0.9996 \times 150 + 0.0004 \times 5\,000\,000 = 149.94 + 2000 = 2149.94 \text{ ns} \approx 14.3\, t_m.$$

Slow-down below 20%: EAT < 180 ns, so $150 + p\,(5\,000\,000 - 150) < 180 \Rightarrow p < 30/4\,999\,850 \approx 6.0 \times 10^{-6}$, at most one fault per about $166\,700$ accesses.

(b) $N = 4$: $1/(0.3 + 0.7/4) = 1/0.475 \approx 2.11$; $N = 10$: $1/(0.3 + 0.07) = 1/0.37 \approx 2.70$; limit $1/0.3 \approx 3.33$. *Rubric:* (a) 6 (3 EAT, 3 bound), (b) 4.

13 How the Numbers Were Checked

Every number in this guide, on the slides and in the sample paper was computed, not eyeballed. The `code/` folder contains:

File	Purpose
`osim.py`	small simulators with exactly the course conventions: FCFS, SJF, SRTF, Priority, RR; FIFO, LRU, OPT; first/best/worst-fit; the formulas of Weeks 1–7
`review_problems.py`	the slides' worked problems and the examples of Sections 2–8 (output: `out_review.txt`)
`practice_exam.py`	the practice exam of Section 11 (output: `out_practice.txt`)
`sample_exam.py`	the separate sample paper and its answer key (output: `out_sample.txt`)
`gen_tables.py`	writes the Gantt charts and tables marked "generated" (`tex/*.tex`)

For example, the SRTF part of Problem 2, as printed by `python3 review_problems.py`:

```
--- SRTF ---
Gantt: [0-2 P1] [2-3 P2] [3-5 P3] [5-8 P2] [8-13 P1] [13-18 P4]
      arr burst prio first done  TAT wait resp
P1      0     7    2     0   13   13    6    0
P2      2     4    1     2    8    6    2    0
P3      3     2    4     3    5    2    0    0
P4      5     5    3    13   18   13    8    8
sum waiting 16, sum turnaround 34, sum response 8
avg waiting 4  avg turnaround 8.5  avg response 2  context switches 5  CPU busy 18/18
     ms
```

Change the job sets, reference strings or holes in the scripts to generate your own practice problems, and check your hand calculations against them.

14 Looking Ahead: Week 9

After the midterm the course becomes hands-on. From Week 9 to Week 14 we work in the Linux shell every week, and many of this half's ideas return as commands.

- **Week 9: Linux Shell Basics & Filesystem Navigation.** Getting Linux (lab PCs, WSL with `wsl --install`, an Ubuntu VM); terminal versus shell versus kernel (the shell is a system program, Week 2); first commands; `pwd`, `ls`, `cd`; absolute and relative paths in the directory tree of Week 7; getting help.
- Week 10: file operations and links (inodes again); Week 11: permissions; Week 12: redirection and pipes; Week 13: processes and job control (`ps`, signals; Weeks 3 and 5); Week 14: shell scripting; Week 15: final exam.

Before Week 9, make sure you can open a Linux terminal on a lab PC, in WSL or in a virtual machine.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Chapters 1–2 (introduction, structures), 3 (processes), 4 (threads), 5 (CPU scheduling), 6 (synchronisation tools), 9 (main memory), 10 (virtual memory), 13 (file-system interface).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, chapter PDFs:
  - Processes: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf`
  - Scheduling: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf`
  - Concurrency and threads: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf`
  - Paging: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-paging.pdf`
  - TLBs: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-tlbs.pdf`
  - Swapping policies: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf`
  - Files and directories: `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`